AI/ML PORTFOLIO PROJECT | COMPUTER VISION

AI-Based Student Face Recognition System

Smart Attendance: face-based period attendance, reports and a student portal

Face Recognition | Deep Learning | Flask | SQLite



Conceptual graphic (not a screenshot)

Thangaraj T | tmthangaraj1980@gmail.com | +91 9952045978
github.com/tmthangaraj1980-AIML

Status: source-code reviewed. No repository URL or demo recording supplied for this project.

Executive overview

- Staff enroll a student with one webcam capture; a 512-D face vector is stored
- During a class period the scanner detects every face and matches it to enrolled students
- Present students are written to an attendance register for that period (P1 to P8)
- Daily, range and shortage (<75%) reports export to PDF and Excel
- Students sign in to a portal to see their own attendance

512

dimensions per face embedding

0.58

cosine-similarity match threshold

8

teaching periods per day supported

1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

Problem statement

Time lost

Roll call every period takes minutes from teaching time.

Error-prone records

Hand-written registers need re-keying; totals and percentages are easy to miscount.

Slow reporting

Daily registers, cumulative reports and shortage lists are compiled by hand.

Low student visibility

Students rarely see their running percentage until a shortage is declared.

1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

Objectives and use cases

Objectives

- Automate period attendance
- Store one face template per student
- Prevent duplicate marks per period
- Generate registers and shortage lists
- Give students self-service access

Staff user

- Log in as staff
- Enroll students
- Start a period session (date, period, subject, class)
- Run the live scanner
- Export PDF / Excel reports

Student user

- Log in with roll number
- See today's periods 1 to 8
- See subject-wise totals
- See overall percentage
- Get a warning below 75%

1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

Technology stack

Web application

- Python, Flask 2.3+ (routes, sessions)
- Jinja2 HTML templates, Bootstrap-style UI
- MJPEG live video streaming

Computer vision and deep learning

- OpenCV (capture, CLAHE, drawing)
- facenet-pytorch: MTCNN + InceptionResnetV1
- PyTorch / torchvision, Pillow, NumPy

Data and configuration

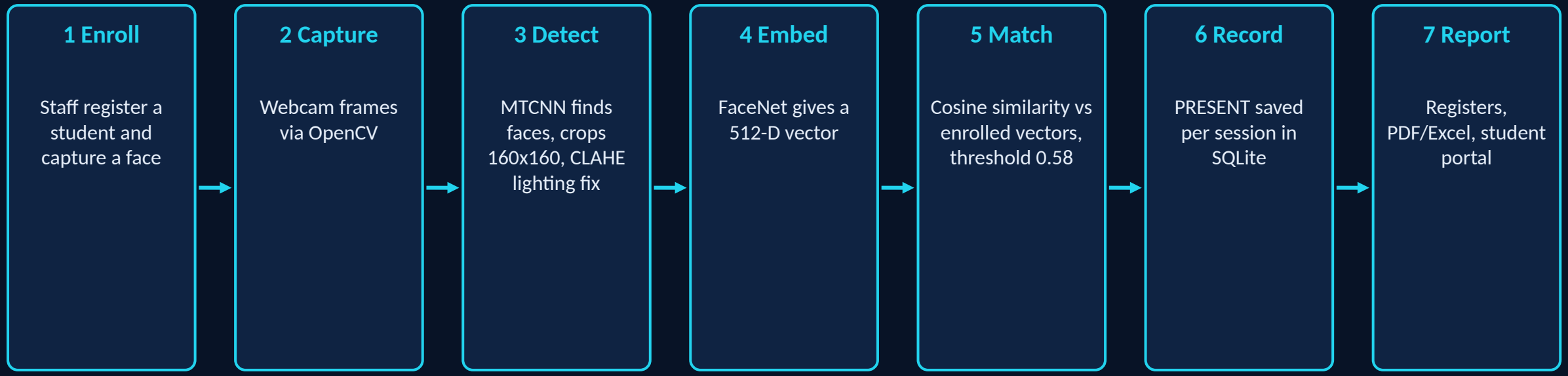
- SQLite (students, sessions, records, users)
- NumPy .npz store of face embeddings
- YAML config (thresholds, paths)

Reporting

- ReportLab for PDF registers
- openpyxl for Excel export
- Daily CSV log in the desktop scanner

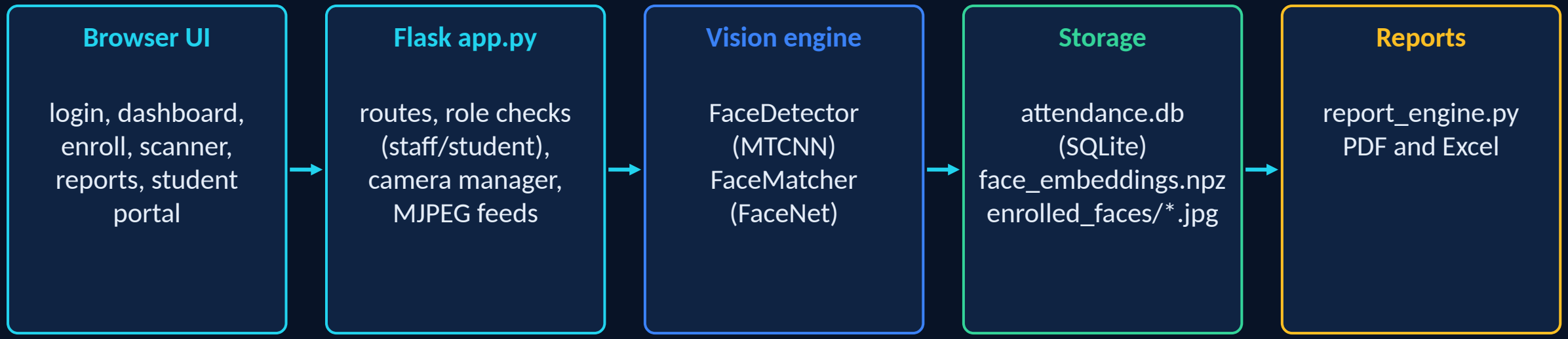
1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

End-to-end project flow



Enrollment runs once per student. Stages 2 to 6 repeat for every frame of every class period. Stage 7 reads the saved records.

System architecture



Second entry point: desktop scanner (main.py)

OpenCV window with FPS, face boxes and status. Adds a 5-frame voting buffer, a 300 s cooldown and logs to a SQLite attendance table plus a daily CSV.

1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

Stage 1: Student enrollment

Enrollment form

roll no, name, degree, dept, year, sem, section, password

→

Camera preview

MJPEG feed with "Align Face Here" box

→

Capture 1 frame

camera manager reads a frame

→

detect_and_crop

largest face, CLAHE, normalised tensor

→

Save

vector to .npz, JPG, student + user rows

Rules in code

- Roll number and name are required
- No face found: request rejected
- Roll number is the vector key

Gap to note

- Config lists 3 to 5 photos
- Web route uses one capture

Stages 2 and 3: capture and face detection

DynamicCameraManager

thread-safe, ref-counted, 1280x720

→

Frame (BGR)

converted to RGB / PIL

→

MTCNN detect

boxes + probabilities, all faces

→

Filter

keep prob >= 0.60

MTCNN setting	Value
min_face_size	20 px
thresholds (P/R/O-Net)	0.6 / 0.7 / 0.7
scale factor	0.709
margin around box	20 px
keep_all	True (multi-face)

Why this matters

- Detection answers "where are faces?"
- Recognition answers "whose face?"
- Multi-face support lets one frame cover a whole row of students

Preprocessing: crop, resize, lighting normalisation



Design choice

- The same preprocessing is applied at enrollment and at scan time, so stored and live vectors are comparable
- CLAHE targets uneven classroom lighting, one of the main failure causes for face recognition

Stage 4: face embedding with FaceNet



What an embedding is

- A point in 512-D space
- Same person: points close together
- Different people: points far apart

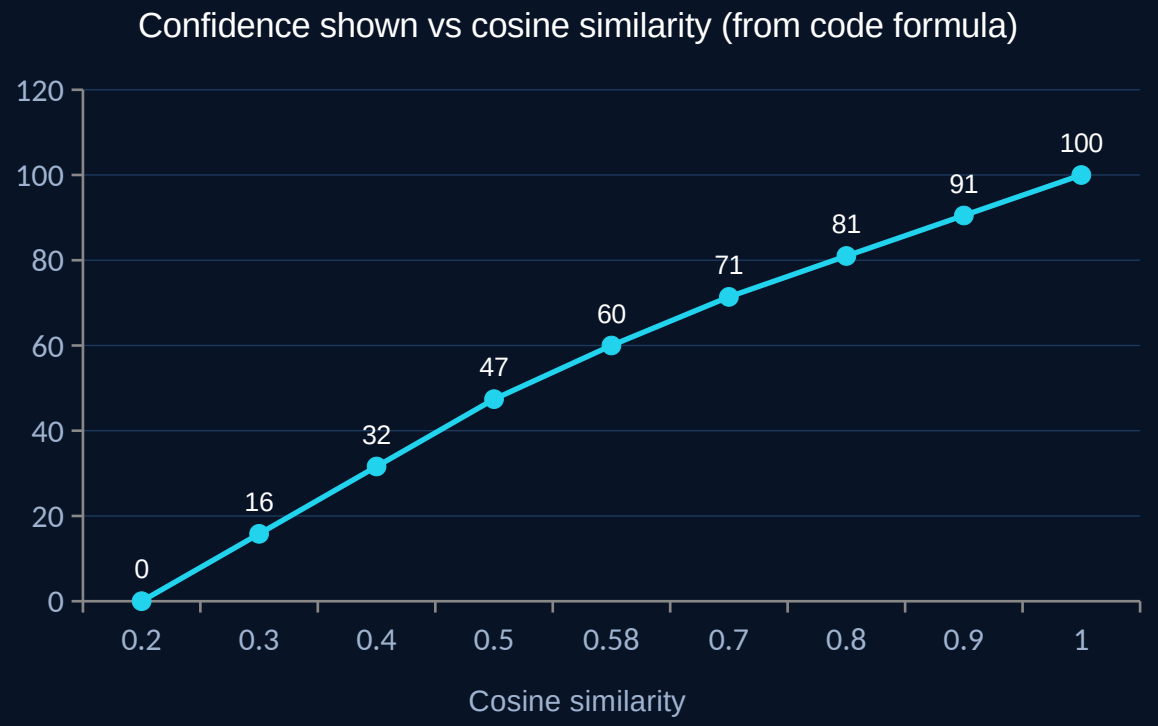
Model provenance

- Pretrained, not trained here
- Inference only (classify=False)
- Accuracy depends on this deployment, not on the paper

Stage 5: matching with cosine similarity

Matching steps

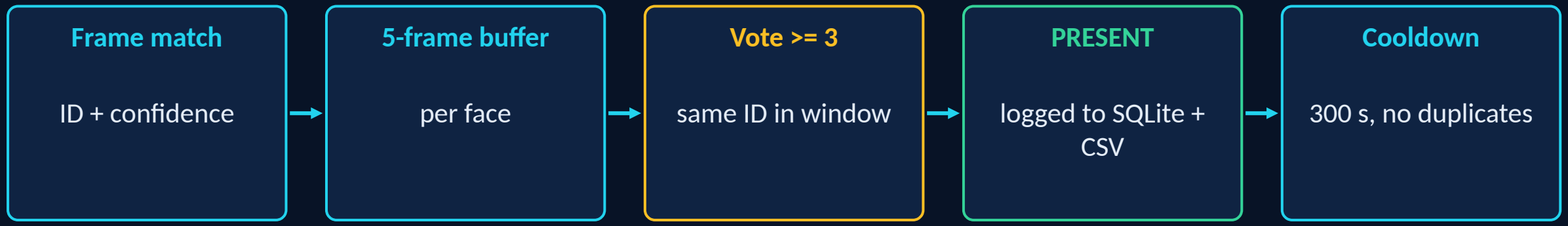
- Embed the live face (unit vector)
- Dot product with every enrolled vector
- Pick the highest similarity
- >= 0.58: identity accepted
- < 0.58: face stays Unknown
- Linear search: fine for a class, slow for thousands



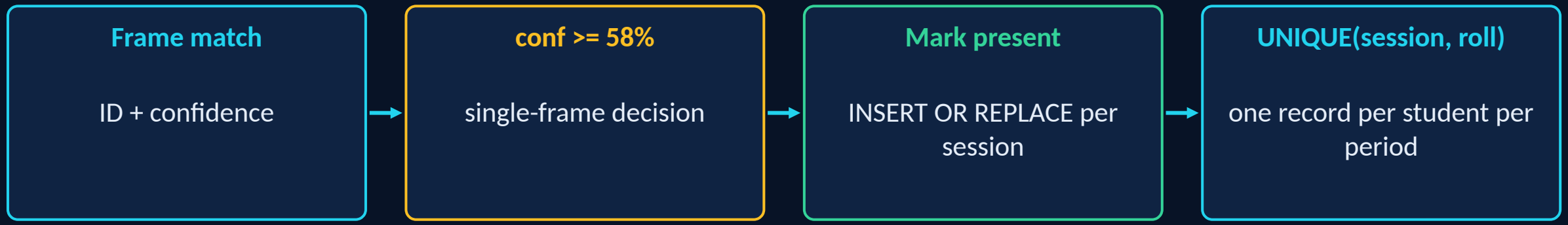
The 60% display point sits exactly at the 0.58 threshold.

Stage 5 to 6: decision logic and duplicate control

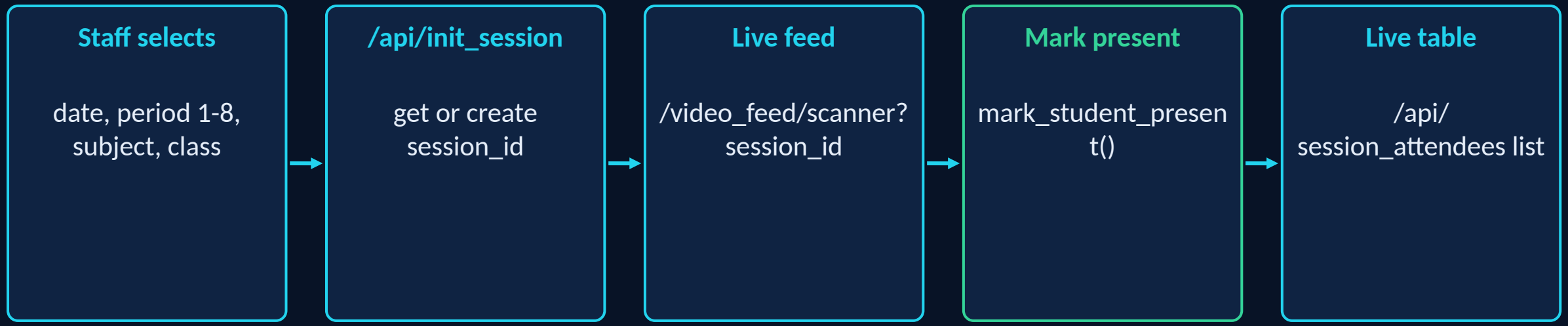
Desktop scanner (main.py + attendance_tracker.py)



Web scanner (app.py)



Stage 6: period session and live scanner



Session key

Date + period + degree + department + year + semester + section. Subject code and staff name are stored on the session for later reporting.

Data model (SQLite)

students

- roll_number (PK)
- name, degree, dept
- year, sem, section

attendance_sessions

- session_id (PK)
- date, period
- subject, staff, class

attendance_records

- session_id (FK)
- roll_number
- timestamp, confidence, status

users

- username, password
- role: staff / student
- full_name

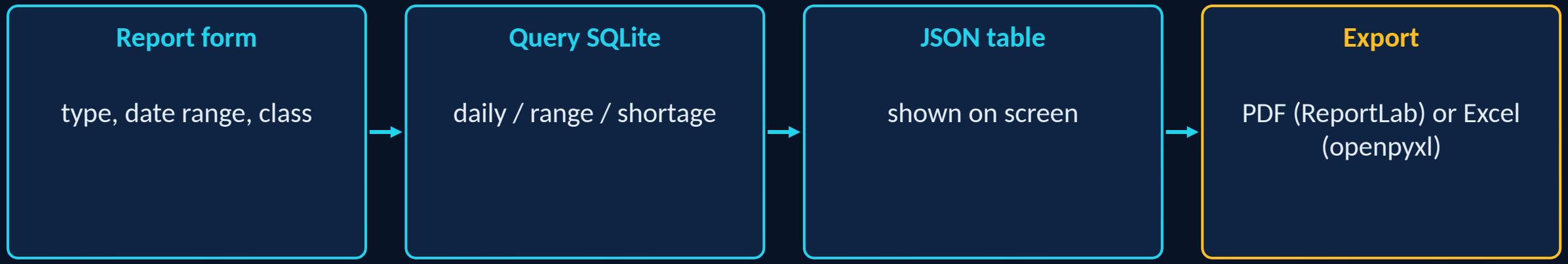
Biometric store

face_embeddings.npz holds one 512-D vector per roll number. Enrolled JPGs are saved to data/enrolled_faces/.

Legacy log

attendance(student_id, timestamp, confidence, status) is written by the desktop scanner, alongside daily CSV files.

Stage 7: reports and exports



Daily register

- Student x P1..P8 matrix
- Attended and day %

Range report

- From / to dates
- Cumulative attendance

Shortage report

- Percentage below 75%
- Lists at-risk students

Stage 7: student portal

Student login

roll number + password

→

Role check

student_required decorator

→

Portal data

get_student_portal_data()

→

Dashboard

periods, subjects, overall %

Shows

- Today status for periods 1 to 8
- Subject-wise held / attended / %
- Overall percentage

Alerts

- Shortage warning below 75%
- Colour-coded percentage

Application screens (schematic of the scanner page)



1. Controls: session settings
2. Live camera + face boxes
3. Live attendee table

Green box: matched at or above threshold. Red box: Unknown or still verifying.

Illustrative schematic, not a real screenshot

Verified configuration and evidence

Parameter	Value (config.yaml)
Detector	MTCNN, face 160x160, margin 20, min prob 0.60
Recognizer	InceptionResnetV1, vggface2, 512-D
Match threshold	cosine similarity 0.58
Voting (desktop)	3 matches in a 5-frame window
Duplicate cooldown	300 seconds
Shortage rule	below 75% attendance

Supplied data files

- 2 enrolled vectors in the .npz
- 6 sessions, 10 records in DB
- Sample / test data only

Not measured

- Recognition accuracy
- FAR / FRR
- Per-frame latency

1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

How accuracy should be evaluated

Metric	Meaning	How to measure here
FAR	Unknown or wrong person accepted	Impostor attempts above 0.58 / all impostor attempts
FRR	Enrolled student rejected	Genuine attempts below 0.58 / all genuine attempts
Top-1 accuracy	Correct identity returned	Labelled class photos, per-student
EER / threshold sweep	Where FAR equals FRR	Vary threshold 0.40 to 0.80
Conditions	Robustness	Lighting, angle, glasses, distance

Report results per condition, not as a single number.

Technical challenges and limitations

Recognition

- One enrollment frame per student
- No liveness / anti-spoofing
- Threshold 0.58 not yet calibrated

Tracking

- Face buffers keyed by list index
- Web scanner marks on one frame

Security

- Passwords stored in plain text
- Default admin account and static secret key
- Debug off, but host 0.0.0.0

Packaging

- openpyxl missing from requirements
- FaceNet loaded twice in matcher
- Linear match search

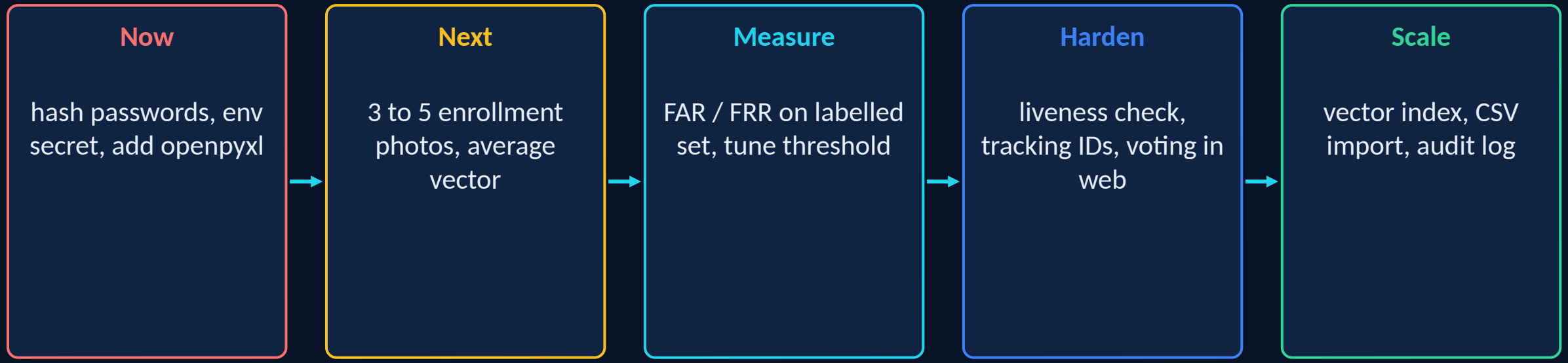
Privacy, consent and responsible use



Also consider

- Bias: measure error rates across skin tones, ages and lighting
- Unknown faces are not logged, which limits data collection

Improvements and roadmap



Interview preparation

1. Did you train the face model?

No. MTCNN and InceptionResnetV1 (VGGFace2 weights) are pretrained. I built detection-to-database pipeline, matching and reports.

2. Why cosine similarity?

Embeddings are L2-normalised, so a dot product gives cosine similarity; the 0.58 threshold separates match from unknown.

3. Detection vs recognition?

Detection locates faces (MTCNN). Recognition decides identity by comparing embeddings (FaceNet + matcher).

4. How do you avoid false marks and duplicates?

Threshold, 3-of-5 voting and 300 s cooldown on desktop; UNIQUE(session, roll) in the database for web.

5. How would you evaluate accuracy?

Labelled genuine and impostor pairs, sweep the threshold, report FAR, FRR, EER per lighting and angle condition.

1 Enroll | 2 Capture | 3 Detect | 4 Embed | 5 Match | 6 Record | 7 Report

Resources and missing details

Available

- Source: smart_attendance.zip (reviewed)
- Profile: github.com/tmthangaraj1980-AIML
- No project repo URL confirmed
- No live demo link

github.com/tmthangaraj1980-AIML

Missing or unverified

- Demo recording or real screenshots
- Recognition accuracy, FAR, FRR
- Number of real enrolled students
- Camera hardware and room setup
- Deployment status in a college

Conclusion

Skills demonstrated

- Computer vision pipeline design
- Pretrained deep-learning inference
- Flask full-stack with role access
- SQLite schema and reporting
- Honest evaluation and risk thinking

Project impact

- Replaces manual roll call with a scan
- Produces registers and shortage lists
- Gives students self-service visibility
- Clear path to a measured, secure v2

Thangaraj T | tmthangaraj1980@gmail.com | +91 9952045978